

Marwadi Education Foundation's Group of Institutions
Faculty of Computer Applications
MCA Sem- III

Fundamental of Java Programming
(630002)



Unit – 1
Basics of Java



Introduction to Java

Java is a general purpose object oriented Programming language. It is developed by Sun Microsystems of USA in 1991.

Its original name was **Oak** given by **James Gosling**, who is one of the inventors of the language.

Java was initially Developed for Softwares for Consumer Electronics Devices like TVs, VCRs, and others.

The father of Java – James Gosling



Introduction to Java

Java has taken the most popular languages of that time i.e. C and C++ as base by considering its various features and removing their limitations.

With this they have develop a most simple, reliable, portable and powerful language.

History of Java

Year	Development
1990	Sun Microsystems decided to develop a special software for consumer electronics devices. A Team has been formed to undertake this task. James Gosling was the head of that team.
1991	The team announce a new language called “Oak”
1992	The team known as “Green Project” team, have demonstrated the use of language on a list of home appliances.
1993	World Wide Web (WWW) has given support to Green Project Team and they have started thinking for development of Web Applets
1994	A new Web browser called HotJava has been developed by the Team to run applets.
1995	Oak was rename to Java due to some legal problems.
1996	Sun release Java Development Kit 1.0 (JDK 1.0)

History of Java

Year	Development
1997	Sun release JDK 1.1
1998	Sun release Java 2 with JDK 1.2
1999	Sun release J2SE and J2EE
2000	JDK 1.3
2002	JDK 1.4
2004	JDK 1.5
2006	JDK 1.6

Features to Java

Java has various features which makes it simple, secure and compact. They are as follows:

- (1) Compiled and Interpreted
- (2) Platform Independent and Portable
- (3) Object Oriented
- (4) Secure
- (5) Distributed
- (6) Familiar, Simple and Small
- (7) Multithreaded and Interactive
- (8) High Performance
- (9) Dynamic and Extensible

Features to Java

(1) Compiled and Interpreted :

Usually a computer language is either compiled or interpreted. But java combines both the approaches. First java compiler translates the source code into byte code instructions. Bytecodes are not machine instructions so in second stage, Java Interpreter generates machine code and execute the code. So we can say that java is Compiler and Interpreter.

Phase – I (Compilation)



Phase – II (Interpretation)



Features to Java

(2) Platform Independent and Portable :

Java programs can be moved easily from one system to another, anywhere and anytime. Due to this reason only the java is the most popular language on the internet. Java is Platform independent because it does not generates the machine code but it generates a code for JVM (Java Virtual Machine) so the program can be run on any machine without any problem.

Features to Java

(3) Object Oriented :

Java is pure Object Oriented Programming Language. Almost everything in java is an object. All programs, codes and data always resides inside the objects and classes.

Features to Java

(4) Secure :

Java is a secure language. It has compile time and run time checking for data types. On the other hand java provides the assurance that no viruses will be communicated with applets. One more thing is java does not support pointers so no question of memory address to user.

Features to Java

(5) Distributed :

Java is a language developed for distributed language for creating application on networks. It has ability to share both – data and programs. This will allow the programmers at various remote locations to work together on a single project.

Features to Java

(6) Simple, Small and Familiar :

Java is simple language because many of the features are from C and C++. Again it removes problems from C and C++ like (1) it does not support pointers, (2) Preprocessors (3) goto statement and many others.

Java is small language because it consist of only Six packages.

It is familiar because it looks like C++. In other words we can say “Java is simplified version of C++”.

Features to Java

(7) Multithreaded and Interactive :

Multithreading means handling multiple tasks simultaneously. Means that we can do more than one work at a time. E.g. listening music is one work and download from internet is another work is the example of Multithreading.

Through java we can develop programs for interactive systems like cellphones, lcds etc.

Features to Java

(8) High Performance :

The performance of Java is quite impressive because it uses compilation and interpretation both. It gives a very much high performance to the java programs. Java architecture has also reduce the overheads which also improves the performance.

Features to Java

(9) Dynamic and Extensible:

Java is dynamic language. It is capable of creating new classes, methods and objects.

Java supports functions written in C or C++ also. This is known as native code. This native code will bind with the code dynamically.

Java is Extensible because we can define our own classes also which can be added to the pure java language.

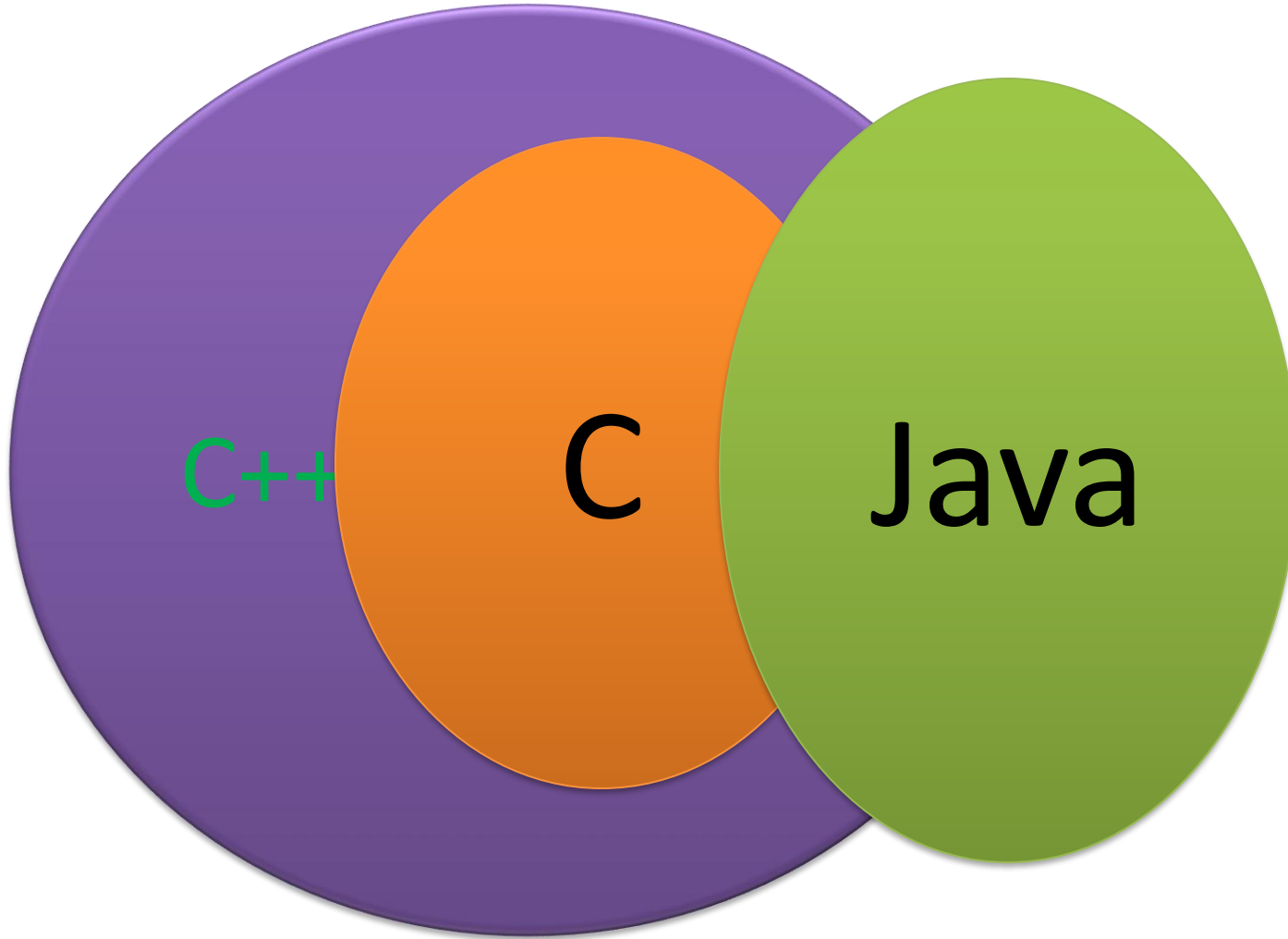
Comparison between C and Java

- (1) Java does not have sizeof or typedef
- (2) Java does not have struct and union
- (3) Java does not have pointers
- (4) Java does not have preprocessors like #include, #define etc.
- (5) Java has new operator **instanceof**
- (6) Java has various features of OOP which are not there in C

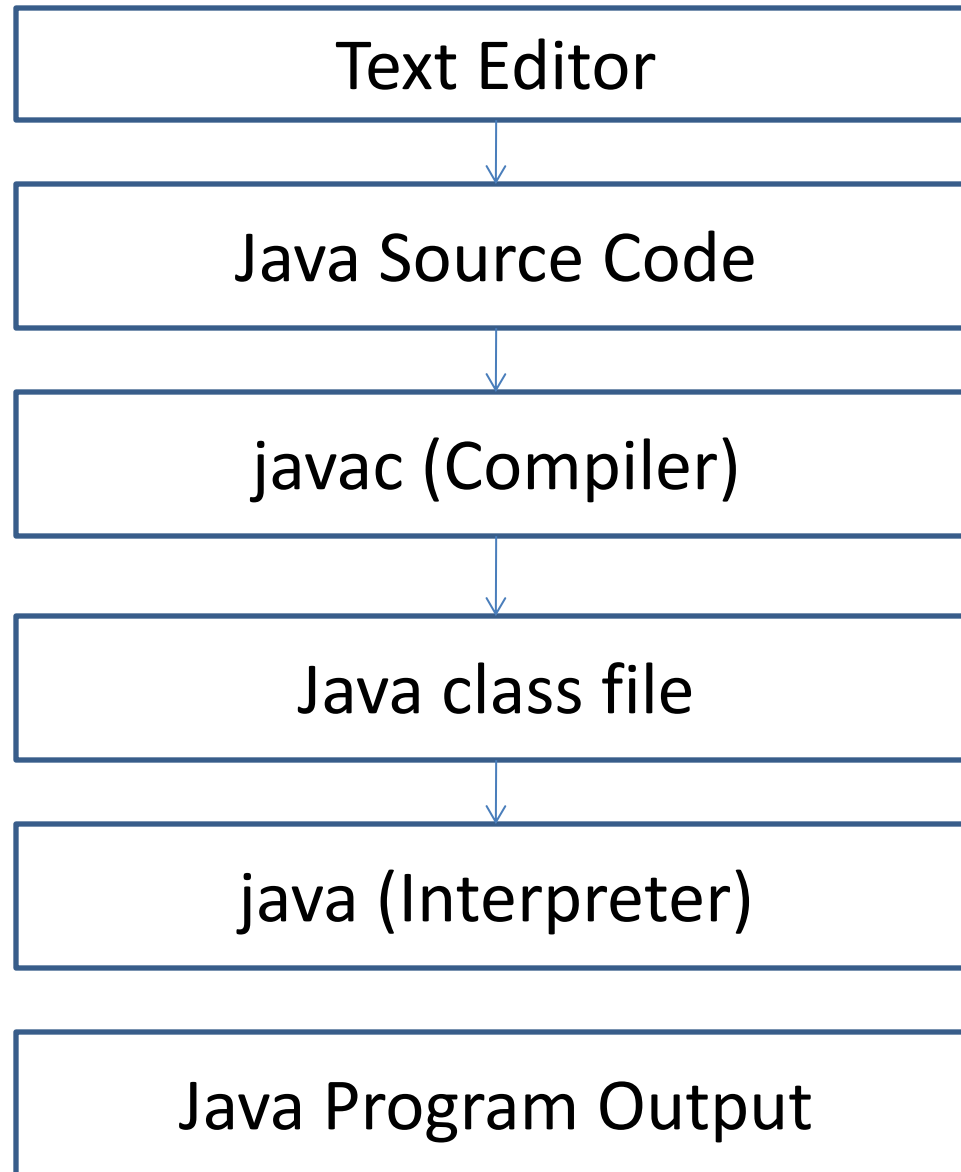
Comparison between C++ and Java

- (1) Java does not support Operator Overloading
- (2) Java does not have template classes
- (3) Java does not support multiple inheritance
instead of that it uses interface
- (4) Java does not support global variable
declaration
- (5) Java does not use destructor instead of that it
uses finalize() method
- (6) There are no header files in java

Summary of C, C++ and Java



Process of Building and Running Java Application



Structure and Basics of Java

Introduction

Java is a general purpose, Object Oriented programming language. We can develop two types of programs :

(1) Stand-alone Applications

(2) Web Applets

We know that to develop a stand-alone application we need two steps

(1) Compile the source code using javac

(2) Execute the byte code using java

Structure of Java Program

Java has specific structure for writing programs (Applications) which is as follows :

- (1) Documentation Section (Suggested)**
- (2) Package Statement (Optional)**
- (3) Import Statement (Optional)**
- (4) Interface Statement (Optional)**
- (5) Class Definitions (Optional)**
- (6) Main Method Class (Compulsory)**

Documentation Section

In Documentation Section we can have the Comments regarding our program which the programmer can refer in the later stage of the application development. We can give comments in three ways

- (1) `/* ..... */` -> Multiple Line Comments
- (2) `//` -> Single Line Comment
- (3) `/** .... */` -> Documentation Comment

Note : Give with example by using **javadoc**

Package Statement

The first statement that is allowed in the java file is package statement. It declares the name of package and inform the compiler that the class defined here is belong to given package name.

We will see about packages later.

Import Statement

The next thing after a package statement (but before the class definition) allowed is the import statement. This is much similar to `#include` statement of C Language.

We see about import later.

Interface Statements

Interfaces are just like a class but it includes a group of methods declaration only. This is also optional element.

We see about Interfaces later.

Class Definitions

In a java program we can have more than one class definitions. The no. of classes used in the program will depend on the complexity of the program.

In other words we can say that the class definition is the backbone of any java program.

Main method Class

This is the most essential element of any java program. Every java stand alone program requires the main method as its starting point. In our simple program we can have the class with main method only.

[ex\ex1.java](#)

[ex\ex2.java](#)

Use below example for comments (javadoc)

[ex\ex3.java](#)

Keywords of Java

Keywords are those words for which the definition is given by java which cannot be changed by the user. Java keywords are as follows :

abstract	assert	boolean	break
byte	case	catch	char
class	const	continue	default
do	double	else	enum
extends	final	finally	float
for	goto	if	implements
import	instanceof	int	interface
long	native	new	package
private	protected	public	return
short	static	strictfp	super
switch	synchronized	this	throw
throws	transient	try	void
volatile	while		

Variable Names Rules and Conventions

Variables are the small piece of data defined by the programmer. In java we have some Rules and some Conventions for declaring Variables.

Rules :

- (1) They can have alphabets, digits, underscore and dollar sign allowed
- (2) They must not begin with digit
- (3) UPPERCASE and lowercase are distinct
- (4) Length is no issue over here

Variable Names Rules and Conventions

Conventions :

- (1) Names of variable in small case when possible
- (2) When more than one words are used in a name the second and subsequent words are in UPPERCASE letters. e.g. dayTemperature, totalMarks etc.
- (3) All classes start with First character as UPPERCASE e.g. Student, Vehicle etc.
- (4) Variable that represents constant value should use UPPERCASE LETTERS and UNDERSCORE BETWEEN WORDS e.g. TOTAL, RATE_OF_INT etc.

Command Line Arguments

Sometimes we need to pass some values while running the program. Here our program will work according to the values given by us at the time of executing the program.

[ex\ex4.java](#)

Basics of Java Programming

Introduction

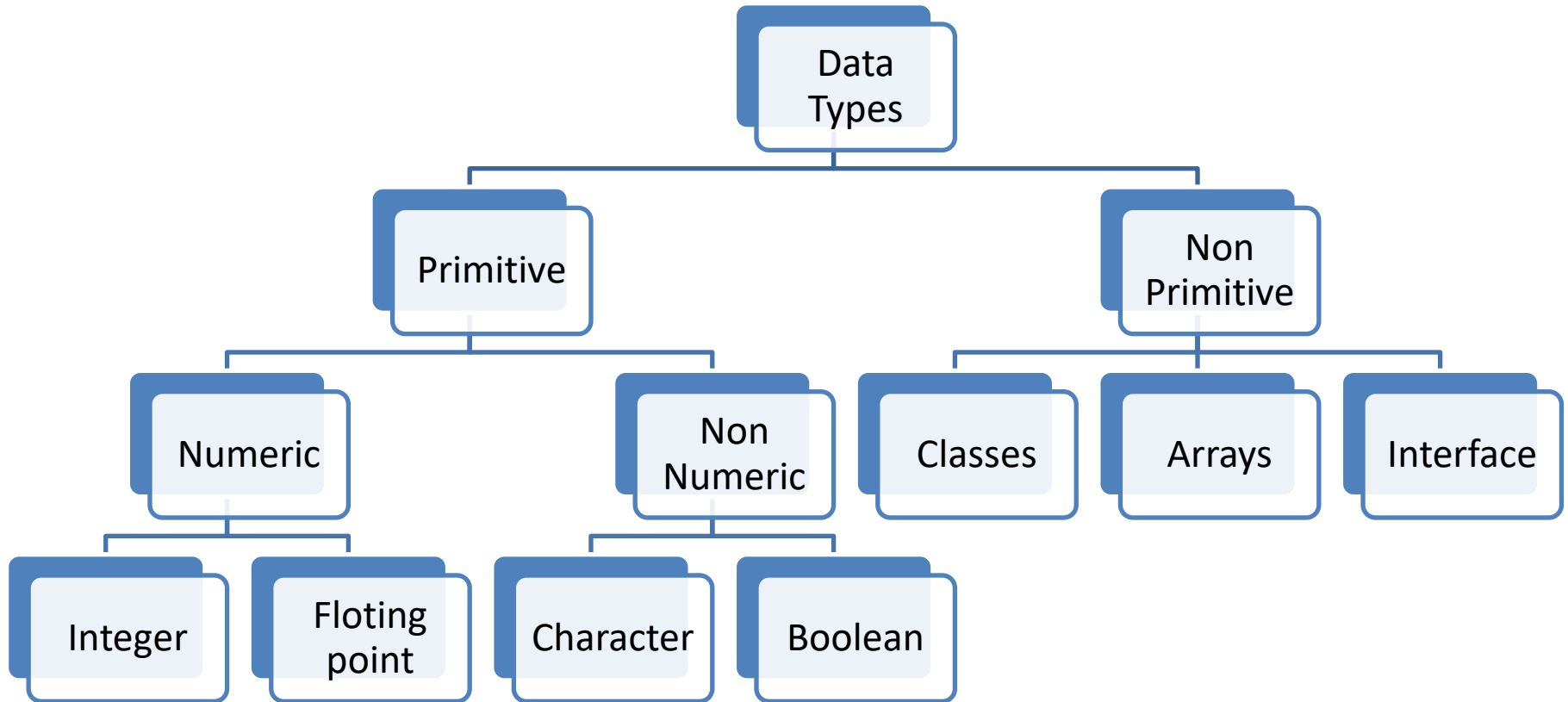
Here, we will discuss about some basic topics of Java Programming. When we need to do some programming we need to understand this topics in detail. The task of processing data is accomplished by executing a sequence of instructions constituting a program.

In programming terminology we are calling it as **syntax**.

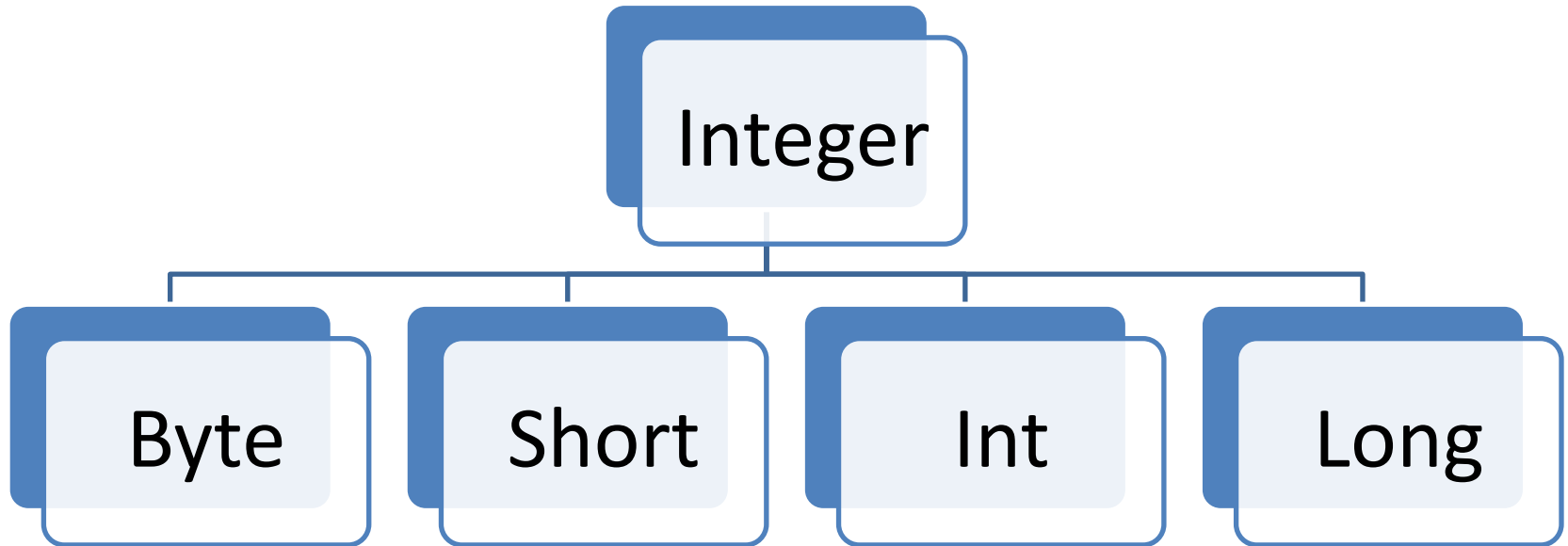
Backslash Codes

Code	Meaning
\b	Back space
\f	Form feed
\n	New line
\r	Carriage return
\t	Horizontal tab
\'	Single quote
\"	Double quote
\\	backslash

Data Types

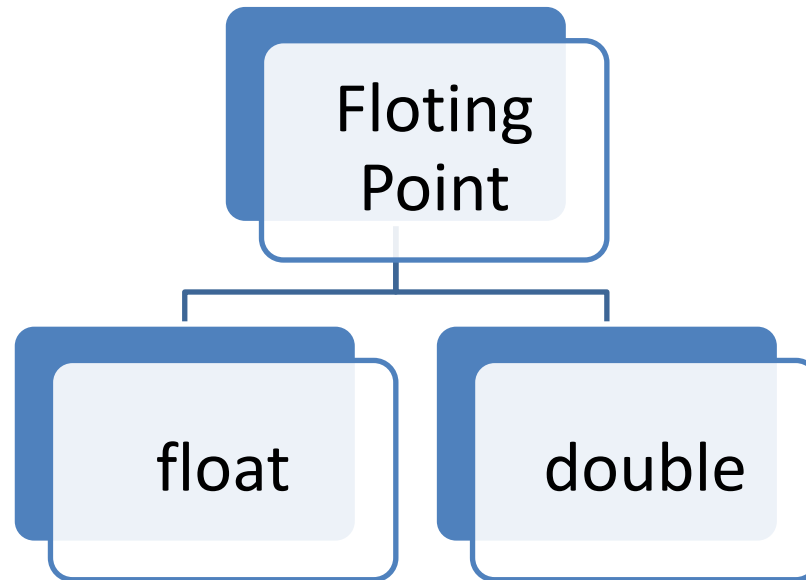


Data Types



Type	Size	Min. Value	Max. Value
byte	One byte	-128	127
short	Two bytes	-32768	32767
int	Four bytes	-2147483648	2147483647
long	Eight bytes	-9223372036854775808	9223372036854775807

Data Types



Type	Size	Min. Value	Max. Value
float	Four bytes	3.4 e -038	3.4 e +038
double	Eight bytes	1.7 e -308	1.7 e +308

Special Note

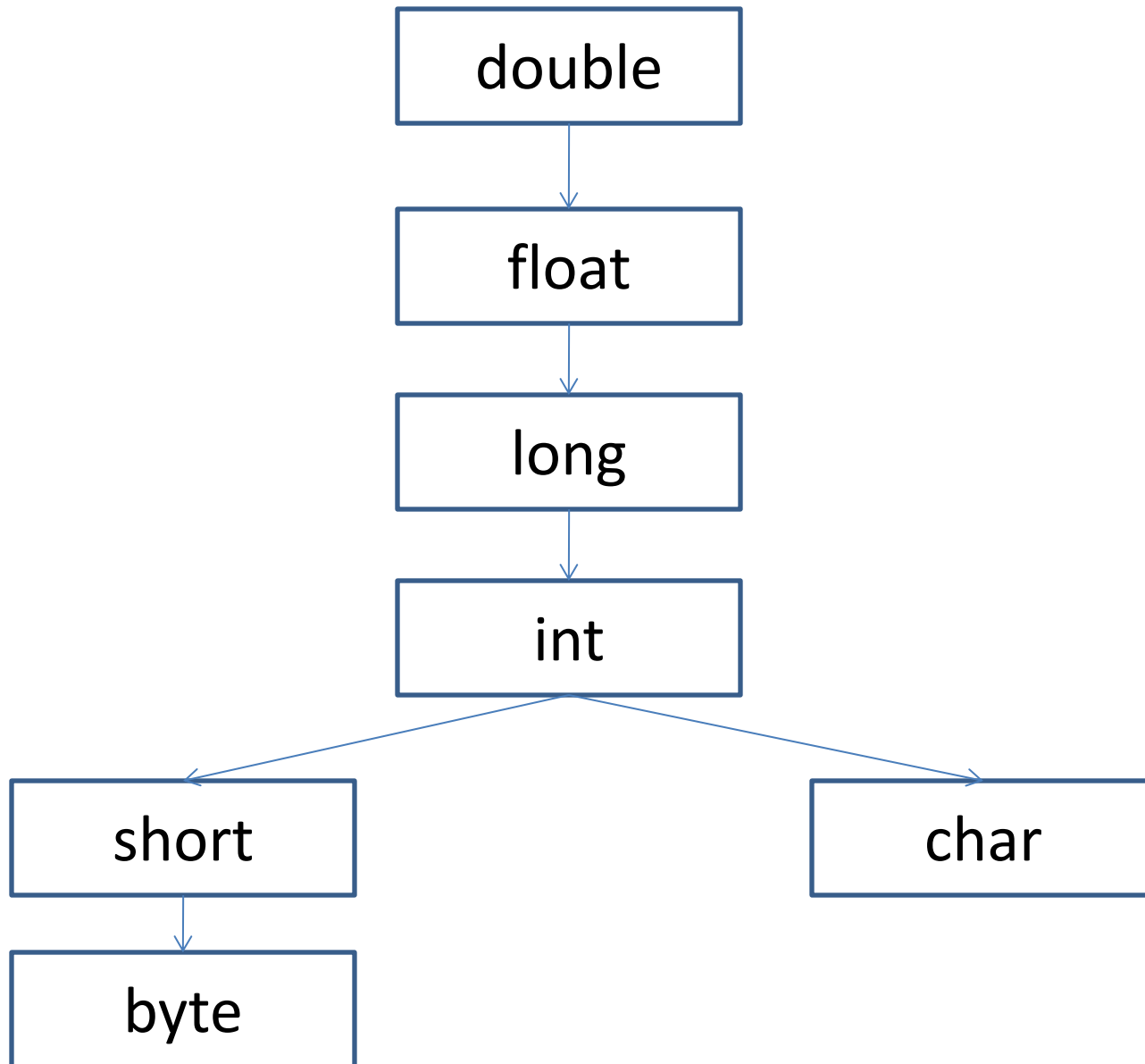
Remember that in Java when we want to enter value of variable through keyboard then at that time we must have to initialize the variable using some value.

[ex\ex5.java](#)

Super Types and Sub Types

Java has defined a relationship between data types. We have a relationship of super-type and sub-type between the data types. A data type is a sub-type of given data type if it is a special kind of the given data type. E.g. in case of primitive data types int is a sub-type of long, as long can always be used wherever int is used. In other words we can say that all ints are long but not all longs may be ints. i.e. we can use super-type for sub-type but we cannot use sub-type for super type.

Super Types and Sub Types



Arithmetic Operators

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulo

Note : in modulo division the sign of answer will be always the sign of first operand means that

$$-14 \% 3 = -2$$

$$-14 \% -3 = -2$$

$$14 \% -3 = 2$$

Modulo division is defined as
 $a \% b = a - (a/b) * b$

Relational Operators

Operator	Meaning
<	Is less than
<=	Is less than or equal to
>	Is greater than
>=	Is greater than or equal to
==	Is equal to
!=	Is not equal to

Logical Operators

Operator	Meaning
&&	Logical AND
	Logical OR
!	Logical NOT

Assignment Operators

Statement with simple assignment operator	Statement with shorthand operator
<code>a=a+1</code>	<code>a+=1</code>
<code>a=a-1</code>	<code>a-=1</code>
<code>a=a*1</code>	<code>a*=1</code>
<code>a=a/1</code>	<code>a/=1</code>
<code>a=a%1</code>	<code>a%=1</code>

Increment / Decrement Operators

Operator	Meaning
++	Increase the value of variable with 1
--	Decrease the value of variable with 1

Conditional Operator

exp1 ? exp2 : exp3

Here, First the exp1 will be evaluated and if the answer is true then exp2 will be evaluated otherwise exp3 will be evaluated

x=a>b ? a : b

Here if the value of a is greater than b then x will be assigned the value of a otherwise the value of b

Bitwise Operators

Operator	Meaning
&	Bitwise AND
!	Bitwise OR
^	Bitwise exclusive OR
~	One's complement
<<	Shift left
>>	Shift right
>>>	Shift right with zero fill

instanceOf operator

Sometimes, knowing the type of an object during run time is useful. For example, you might have one thread of execution that generates various types of objects, and another thread that processes these objects. In this situation, it might be useful for the processing thread to know the type of each object when it receives it. Another situation in which knowledge of an object's type at run time is important involves casting. In Java, an invalid cast causes a run-time error. Many invalid casts can be caught at compile time.

instanceOf operator

For example, a superclass called A can produce two subclasses, called B and C. Thus, casting a B object into type A or casting a C object into type A is legal, but casting a B object into type C (or vice versa) isn't legal. Because an object of type A can refer to objects of either B or C, how can you know, at run time, what type of object is actually being referred to before attempting the cast to type C? It could be an object of type A, B, or C. If it is an object of type B, a run-time exception will be thrown. Java provides the run-time operator instanceof to answer this question.

instanceOf operator

[ex\ex30.java](#)

[ex\ex31.java](#)

Widening Conversion

A widening conversion is the conversion of a sub-type to one of its super types. For numeric types, the following are the widening conversions.

- (1) byte to short,int,long,float and double
- (2) short to int,long,float and double
- (3) int to long,float and double
- (4) long to float and double
- (5) float to double

Narrowing Conversion

A Narrowing conversion is the conversion of a super-type to one of its sub-type. For numeric types, following are the narrowing conversions.

- (1) double to byte, short, int, long and float
- (2) float to byte, short, char, int and long
- (3) long to byte, short and int
- (4) int to byte and short
- (5) short to byte

String Concatenation

+ operator will be used for doing string concatenation.

Remember that String concatenation will be done whenever any one of the two operands is of String type.

e.g.

- (1) $5 + 5 + \text{" is ten"}$ will gives output
"10 is ten"
- (2) $\text{"Fifty five is " + 5 + 5}$ will gives output
"Fifty five is 55"

Some functions from Math Package

Function	Action
<code>sin(x)</code>	returns sin of angle x
<code>cos(x)</code>	returns cos of angle x
<code>tan(x)</code>	returns tangent of angle x
<code>asin(x)</code>	returns the angle whose sine is x
<code>acos(x)</code>	returns the angle whose cosine is x
<code>atan(x)</code>	returns the angle whose tangent is x
<code>pow (x,y)</code>	returns the value of x raised to y
<code>exp(x)</code>	returns the value of e raised to x
<code>log(x)</code>	returns natural log of x
<code>sqrt(x)</code>	returns square root of x

Some functions from Math Package

Function	Action
<code>ceil(x)</code>	returns the nearest whole number less than or equal to x
<code>floor(x)</code>	returns the nearest whole number greater than or equal to x
<code>round(x)</code>	returns the integer closest to the x
<code>abs(a)</code>	returns absolute value of a
<code>max(a,b)</code>	returns maximum out of a and b
<code>min(a,b)</code>	returns minimum out of a and b

Topics as per C and C++

Here we will see the topics which are same as C and C++ in java.

- (1) if statement**
- (2) if – else – if statement**
- (3) for loop**
- (4) while loop**
- (5) do.....while loop**
- (6) switch.....case.....default**

The Enhanced for loop

The enhanced for loop allows you to iterate through a collection without having to create or to calculate beginning and end conditions for a counter variable.

The enhanced for loop is the easiest of the new features to immediately incorporate in your code.

[ex\ex6.java](#)

[ex\ex7.java](#)

Topics as per C++

Here we will see the topics which are same as C++ in java.

- (1) Class and objects**
- (2) member variables and methods**
- (3) constructor**
- (4) method overloading**
- (5) static variables and methods**
- (6) nesting of methods**

[ex\ex8.java](#)

[ex\ex9.java](#)

[ex\ex10.java](#)

Initializer Block

In a class definition, we can have a member block with no name. Such a block is known as the initializer block. An initializer block is never invoked by any application directly since it does not have any name. However, it is always invoked on an instance as soon as it is allocated. It is invoked just before the constructor invoked.

[ex\ex32.java](#)

Class_INITIALIZER Block

Just like we have constructor for initializing the instance variables, we have the class initializer block to initialize the class variables. A class initializer block is created just like the initializer block, but it is declared to be static. We also call this block as the static block.

[ex\ex33.java](#)

Arrays

Introduction

An array is a block of consecutive memory locations that hold values of the same data type. Individual locations are called array's elements. When we say “element” we often mean the value stored in that element.

| 1.39 | | 1.69 | | 1.74 | | 0.0 |



An array of
doubles

Introduction

Rather than treating each element as a separate named variable, the whole array gets one name.

Specific array elements are referred to by using array's name and the element's number, called index or subscript.

1.39	1.69	1.74	0.0
------	------	------	-----

c[0]	c[1]	c[2]	c[3]
------	------	------	------



c is array's
name

Introduction

In Java, an index is written within square brackets following array's name (for example, `a[k]`).

Indices start from 0; the first element of an array `a` is referred to as `a[0]` and the `n`th element as `a[n-1]`.

An index can have any `int` value from 0 to array's length - 1.

We can use as an index an `int` variable or any expression that evaluates to an `int` value.

Introduction

Remember that : In Java, an array is declared with fixed length that cannot be changed.

Java interpreter checks the values of indices at run time and throws **ArrayIndexOutOfBoundsException** if an index is negative or if it is greater than the length of the array - 1

Need for Array

The power of arrays comes from the fact that the value of a subscript can be computed and updated at run time.

No arrays:

1000
times!

```
int sum = 0;  
sum += score0;  
sum += score1;  
...  
sum += score999;
```

With arrays:

```
int n = 1000;  
int sum = 0, k;  
  
for (k = 0; k < n; k++)  
    sum += scores[k];
```

Need for Array

Arrays give direct access to any element — no need to scan the array.

No arrays:

1000
times!

```
if (k == 0)
    display (score0);
else if (k == 1)
    display (score1);
else
    ... // etc.
```

With arrays:

```
display (scores[k]);
```

Arrays as Objects

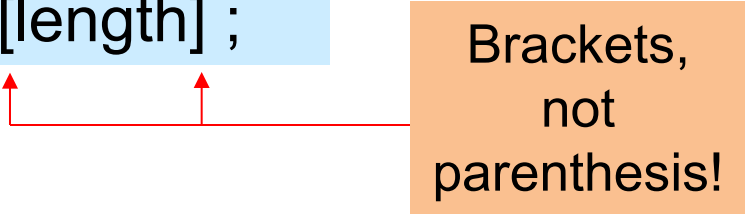
In Java, an array is an object. If the type of its elements is `anyType`, the type of the array object is `anyType[ ]`.

Array declaration:

```
datatype [ ] arrName;
```

One way to create an array:

```
arrName = new anyType [length] ;
```



Brackets,
not
parenthesis!

Declaration and Initialization

In Java, an array is an object. If the type of its elements is `anyType`, the type of the array object is `anyType[ ]`.

Array declaration:

```
datatype [ ] arrName;
```

One way to create an array:

```
arrName = new anyType [length] ;
```



Brackets,
not
parenthesis!

Declaration and Initialization

When an array is created, space is allocated to hold its elements. If a list of values is not given, the elements get the default values. For example:

```
scores = new int [10] ;
```

length 10,
all values
set to 0

```
words = new String [10000];
```

length 10000,
all values set to
null

Declaration and Initialization

An array can be declared and initialized in one statement. For example:

```
int [ ] scores = new int [10] ;
```

```
private double [ ] gasPrices = { 3.05, 3.17, 3.59 };
```

```
String [ ] words = new String [10000];
```

```
String [ ] cities = {"Atlanta", "Boston", "Cincinnati" };
```

Declaration and Initialization

Otherwise, initialization can be postponed until later. For example:

```
String [ ] words;
```

```
...
```

```
words = new String [ console.readInt() ];
```

```
private double[ ] gasPrices;
```

```
...
```

```
gasPrices = new double[ ] { 3.05, 3.17, 3.59 };
```

Length of Array

The length of an array is determined when that array is created.

The length is either given explicitly or comes from the length of the {...} initialization list.

The length of an array `arrName` is referred to in the code as `arrName.length`.

`length` is like a public field (not a method) in an array object.

Unless specific values are given in a {...} list, all the elements are initialized to the default value: 0 for numbers, false for booleans, null for objects.

Length of Array

```
Color[ ] pens;
```

Array not
created yet

```
...
```

```
pens = new Color [ 3 ];
```

Array is created;
all three elements
are set to **null**

```
...
```

```
pens [0] = Color.BLUE;
```

```
pens [1] = new Color (15, 255, 255);
```

```
pens [2] = g.getColor();
```

Two-Dimensional Arrays

2-D arrays are used to represent tables, matrices, game boards, images, etc.

An element of a 2-D array is addressed using a pair of indices, “row” and “column.” For example:

```
board [ r ] [ c ] = 'x';
```

Two-Dimensional Arrays Declaration

```
// 2-D array of char with 5 rows, 7 cols:
```

```
char[ ][ ] letterGrid = new char [5][7];
```

```
// 2-D array of Color with 1024 rows, 768 cols:
```

```
Color[ ][ ] image = new Color [1024][768];
```

```
// 2-D array of double with 2 rows and 3 cols:
```

```
double[ ][ ] sample =
```

```
    { { 0.0, 0.1, 0.2 },  
      { 1.0, 1.1, 1.2 } };
```

Two-Dimensional Arrays Declaration

In Java, a 2-D array is basically a 1-D array of 1-D arrays, its rows. Each row is stored in a separate block of consecutive memory locations.

If `m` is a 2-D array, then `m[k]` is a 1-D array, the k -th row.

`m.length` is the number of rows.

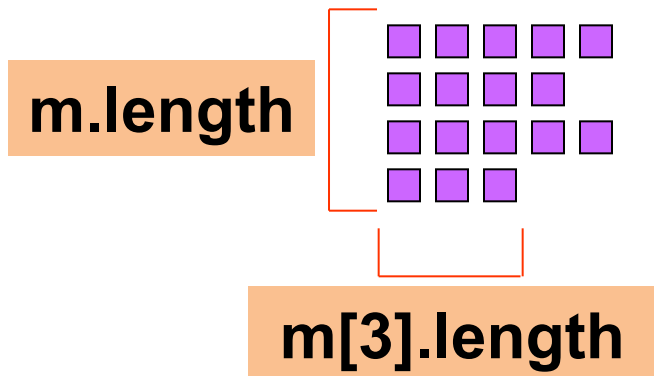
`m[k].length` is the length of the k -th row.

Ragged Array

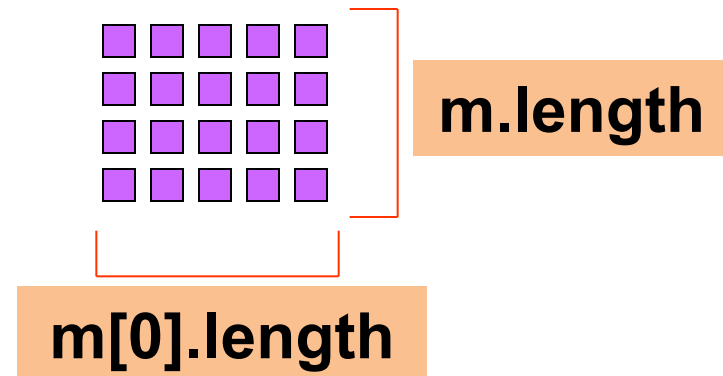
Java allows “ragged” arrays, in which different rows have different lengths.

In a rectangular array, `m[0].length` can be used to represent the number of columns.

“Ragged” array:



Rectangular array:



Ragged Array

```
int ragged[][] = new int[4][];
```

```
for (int i = 0; i < 4; i++) {  
    ragged[i] = new int[i + 1];  
}
```

	0	1	2	3
0	0			
1	10	11		
2	20	21	22	
3	30	31	32	33

```
for (int i = 0; i < 4; i++) {  
    for (int j = 0; j < ragged[i].length; j++) {  
        ragged[i][j] = 10 * i + j;  
    }  
}
```

Ragged Array

```
int[][] tri;  
tri = new int[10][]; // allocate array of rows  
for (int r=0; r<tri.length; r++) {  
    tri[r] = new int[r+1];  
}  
  
// print the triangular array (same as above  
really)  
for (int r=0; r<tri.length; r++) {  
    for (int c=0; c<tri[r].length; c++) {  
        System.out.print(" " + tri[r][c]);  
    }  
    System.out.println("");  
}
```



Thank You